

Python intro

Natural Language Processing for the Digital Humanities

Winter semester 2026/27

Lucija Brozić & Julián Spinelli
Department of Digital Humanities

What is Programming?

- A way to “instruct the computer what to do”
- Collaboration between humans and computer
- Writing code (a set of instructions) to be interpreted by the computer

Problem solving?

“For us, coding is not a set of technical skills but a new type of literacy and personal expression, valuable for everyone, much like learning to write. We see coding as a new way for people organize, express, and share their ideas”

Should You Still Learn to Code in 2026?

"AI has the potential to reshape the entire software and product development life cycle, but realizing that potential requires a foundational change to workflows, roles, and ownership."

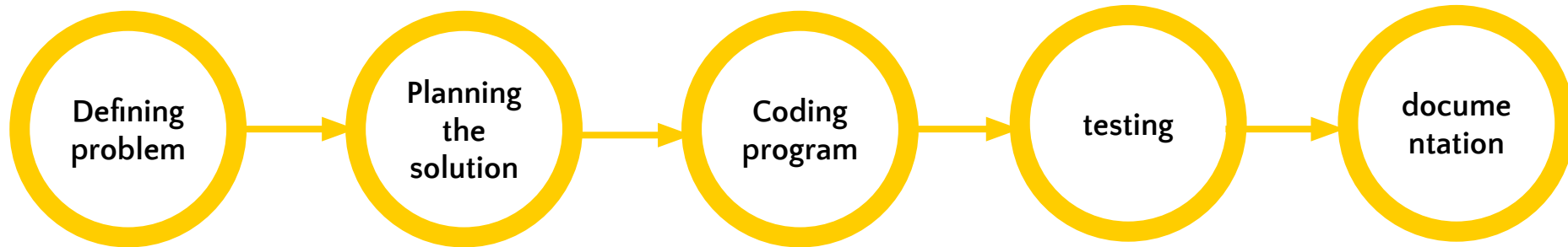
Stack Overflow CEO Prashanth Chandrasekar interview with McKinsey

Problem solving? Programmatic thinking!

"For us, coding is not a set of technical skills but a new type of literacy and personal expression, valuable for everyone, much like learning to write. We see coding as a new way for people organize, express, and share their ideas"

Programmatic - algorithmic thinking

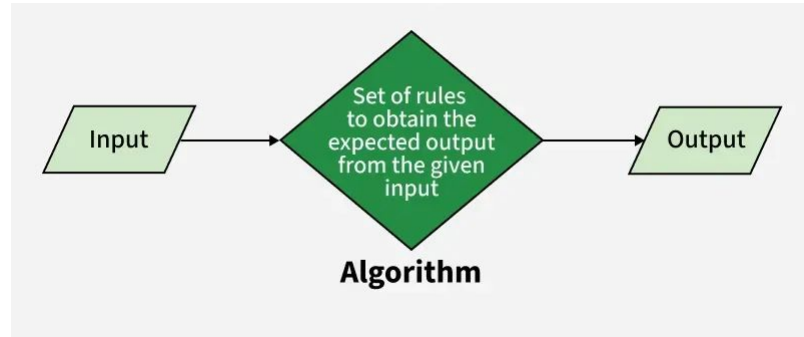
... (often called computational or programmer thinking) is the mental process of breaking down a complex problem into clear, step-by-step logic that a computer can follow to reach a solution.



Algorithm

- *"An algorithm is a well-ordered collection of unambiguous and effectively computable operations that when executed produce a result and halts in a finite amount of time."*
- *"Algorithm is a set of finite, well-defined steps or instructions designed to solve a problem or perform a computation."*

- language independent
- sorting, searching, string processing, numerical problems...

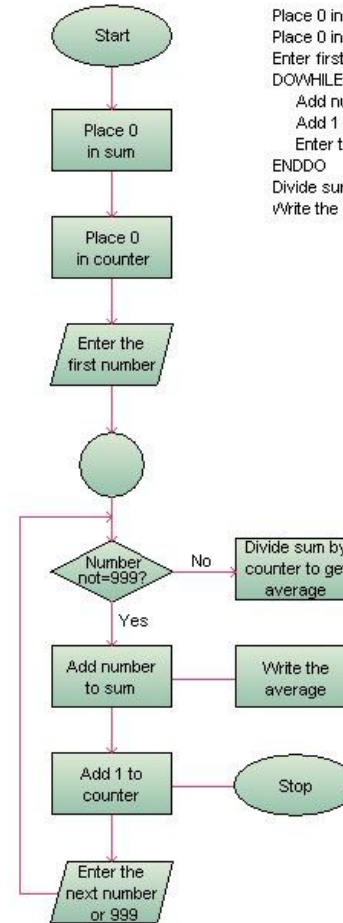


<https://www.geeksforgeeks.org/dsa/introduction-to-algorithms/>

Pseudo code and flow charts

- helps to represent the implementation of an algorithm
- Helps understand sequence of problem solving
- Pseudocode - simplified version of the future code, written in natural language
- Flow chart - diagram that represents a workflow

(a) Flowchart



(b) Pseudocode

```
Place 0 in sum
Place 0 in counter
Enter first number
DOWHILE the number is not equal to 999
    Add number to sum
    Add 1 to counter
    Enter the next number or 999
ENDDO
Divide sum by counter to get ave
Write the average
```

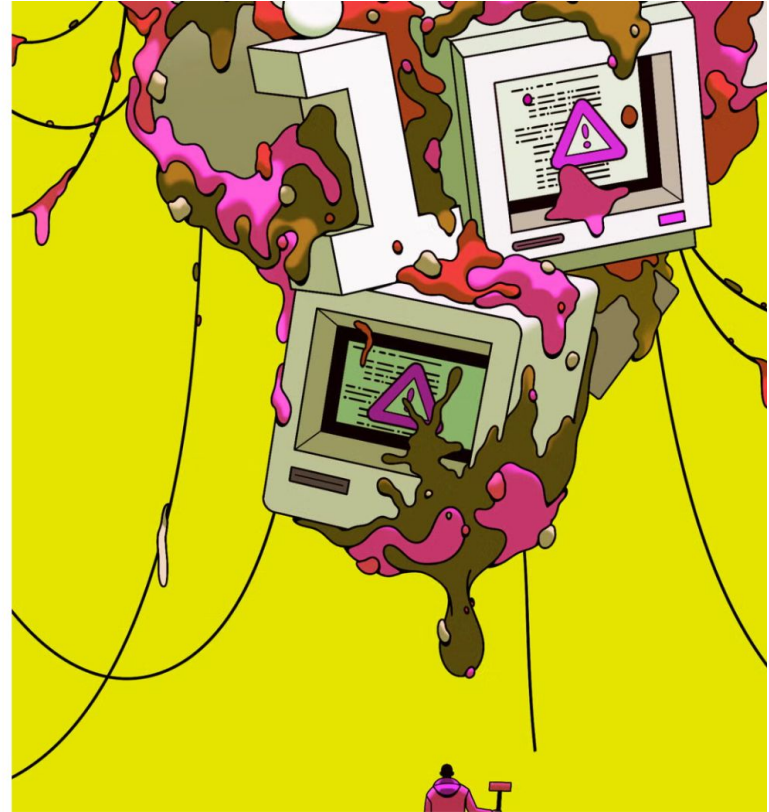
Should You Still Learn to Code in 2026?



Should You Still Learn to Code in 2026?

The Big Bang: A.I. Has Created a Code Overload

Companies are scrambling to deal with the glut.



FROM CODER
(WRITING EVERY LINE)

AI & Software Development

Programmers Aren't Disappearing — They're Mutating: What Coding Really Looks Like in the AI Era



Buddhadev Sasmal

9/22/2026
7 min read

Programmers Aren't Disappearing — They're Mutating: What Coding Really Looks Like in the AI Era

"Five or ten years ago, a developer wrote nearly every line of code by hand. Today, at companies like Intuit, engineers spend far less time on repetitive, boilerplate coding and much more time on system design, edge cases, and the kind of ambiguous business problems AI can't independently reason through ...

The job has shifted from writer to supervisor and editor."

Programmers Aren't Disappearing — They're Mutating: What Coding Really Looks Like in the AI Era

"Traditional programming — the version where a developer's core value was typing correct syntax quickly — is genuinely fading..."

*But "programming" as a discipline — defining problems, architecting systems, making judgment calls about tradeoffs, catching what a machine gets wrong — isn't dying. It's becoming **more valuable**, not less. The demand for people who understand both the fundamentals of software and how to direct AI systems is outpacing the supply, and the salary data already proves it."*

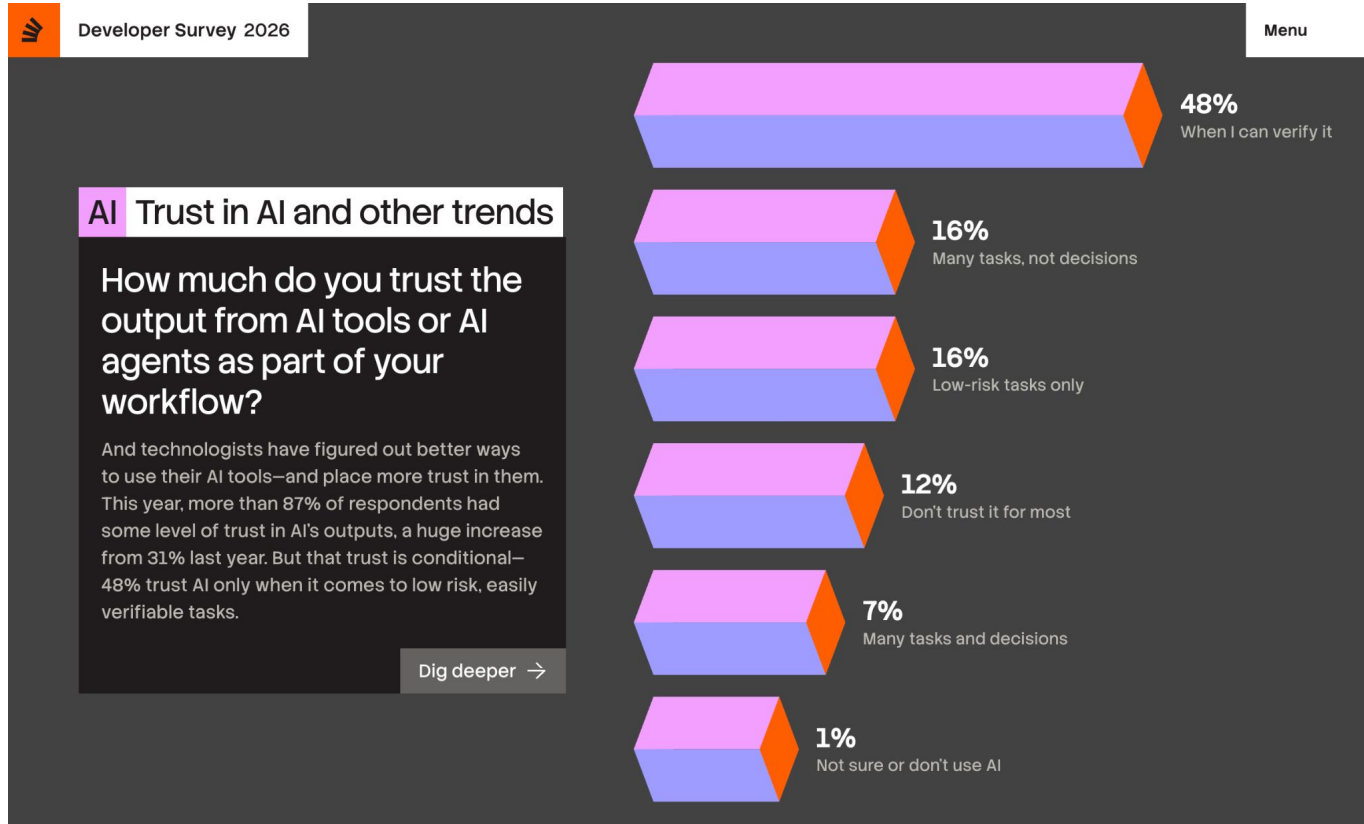
The state of the tech industry in 2026, Gergely Orosz

1. **Speed of change**. In the tech industry, change has never been this large-scale, this fast. Industry legend Martin Fowler confirms.
2. **What's changed**: practically nobody writes code by hand anymore, working with 5-10 agents more common, the fading of the IDE, and 13 more changes from the last year.
3. **What's broken**: assumptions about code output broke, code reviews became theatrical, quality and reliability is down, and more.
4. **What's still the same**: teams are still important, as is planning; non-engineers are still not shipping code, and more.
5. **What comes next?** Cloud coding agents and harnesses, engineers to stop reading the code, companies building a new type of AI infra, and more are already starting, and should accelerate.

https://www.youtube.com/watch?v=Ru99FGJ_yuE

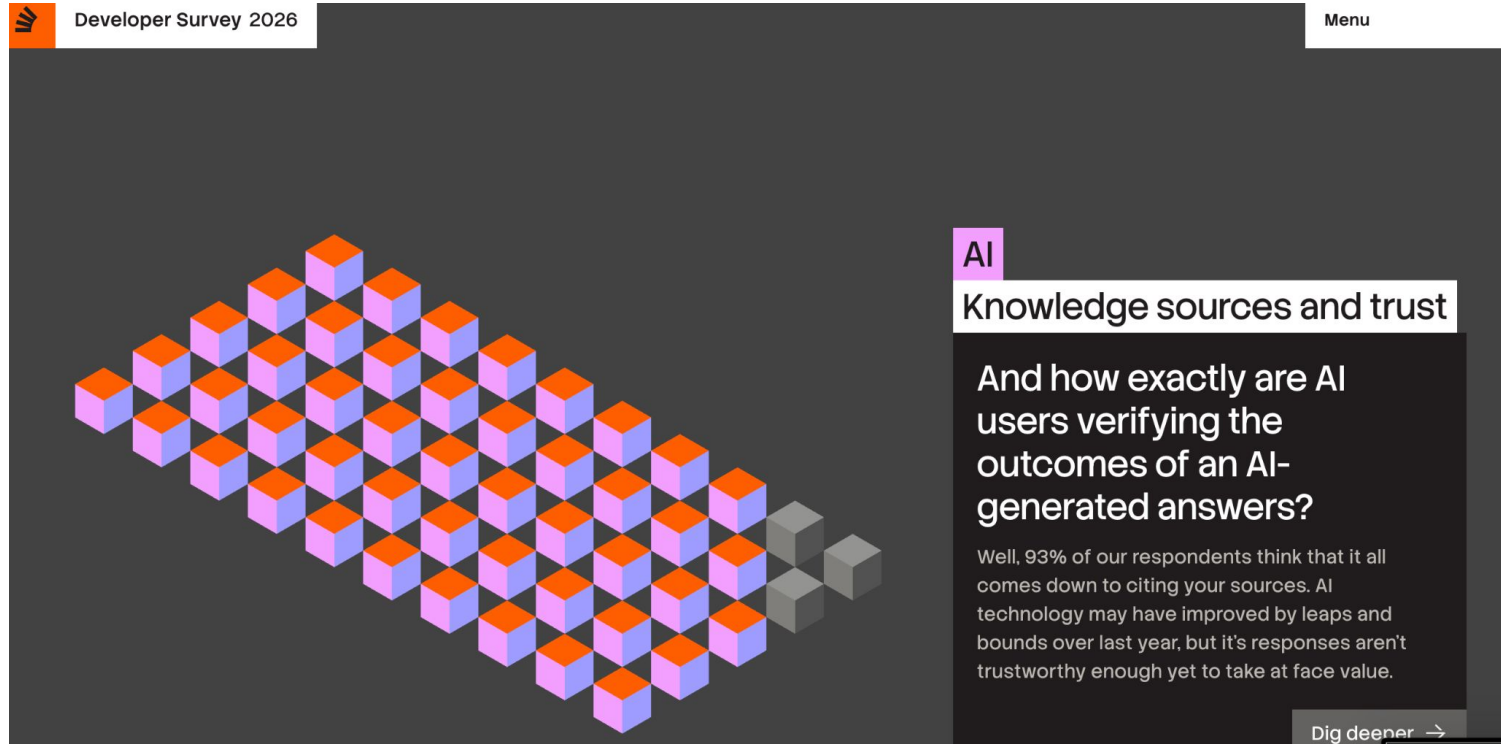
Stack overflow developer survey 2026

<https://survey.stackoverflow.co/2026>



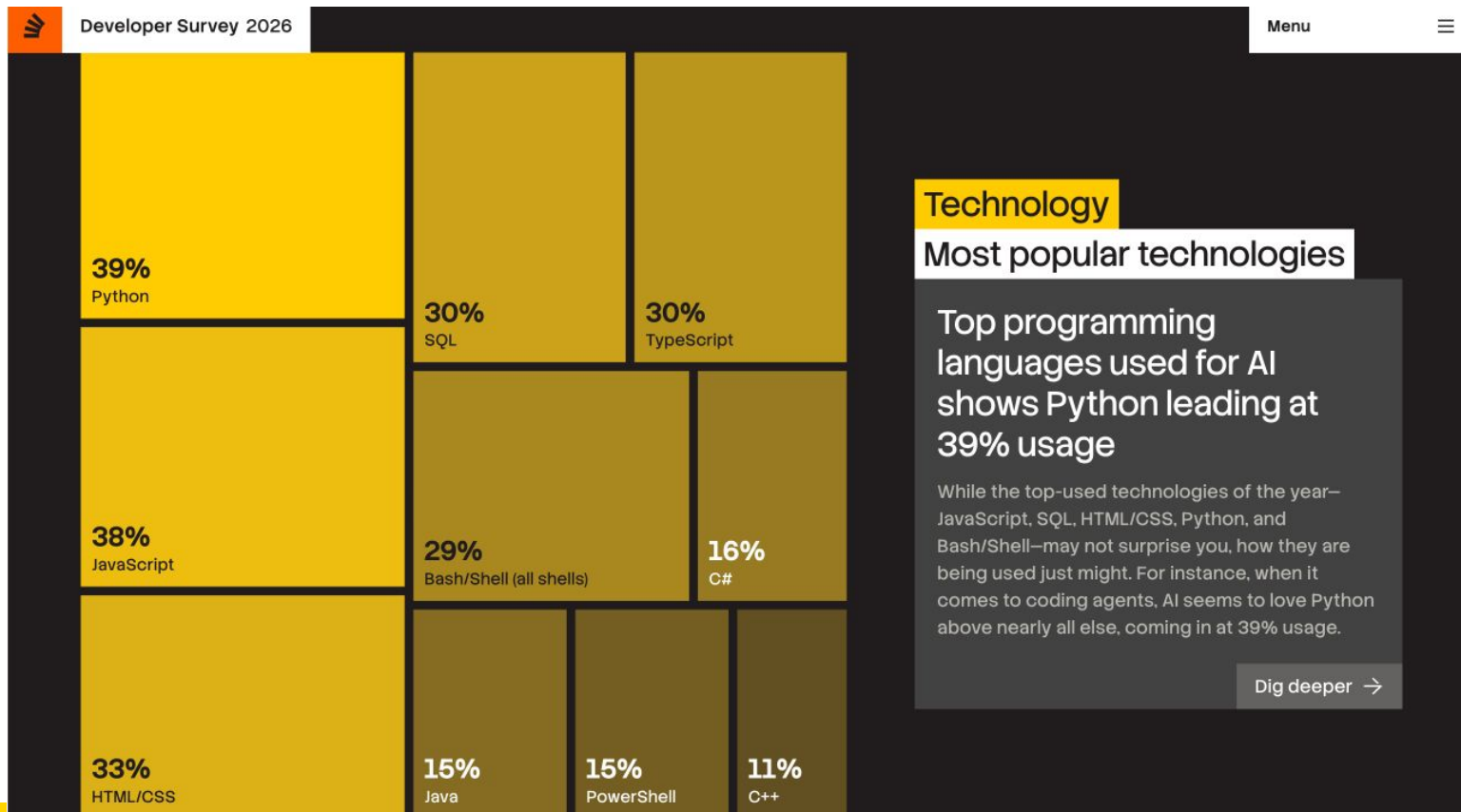
Stack overflow developer survey 2026

<https://survey.stackoverflow.co/2026>



Stack overflow developer survey 2026

<https://survey.stackoverflow.co/2026>



Python



- High level, general-purpose programming language
- Supports multiple paradigms (structured, object-oriented and functional programming)
- Released in 1991
- One of the most popular programming languages, for many usages

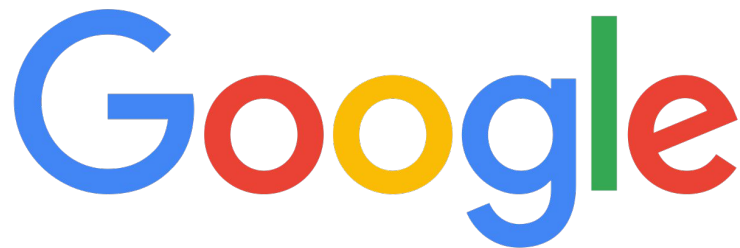
Python



- A lot of modules
- Support libraries (e.g. Numpy for numerical calculations, Django for web development)
- User-friendly data structures
- High-level language
- Easy to read, learn and write

Applications

- GUI desktop apps
- Education
- Web frameworks, development
- data/text manipulation
- web scraping
- NLP
- Data Science
- Machine Learning
- ...



Built in Python: Pinterest, Youtube, Google, Spotify, Facebook, Instagram, Quora, Yelp...

Formal and natural languages

Natural

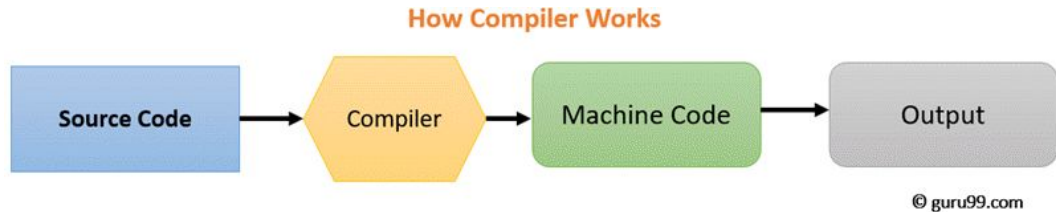
- English, German, Spanish, Croatian, Finnish...
- Not designed by people, evolved naturally
- Full of ambiguity & redundancy
- Not literal

Formal

- Designed by people, for specific applications
- Non-ambiguous, more concise and less redundant, literal, strict syntax rules
- E.g. programming languages

Programming languages by type of translation process

- The translation process converts code written by a programmer into machine code that a computer can execute
- Translating – Compiler (C, C++, Haskell), interpreter (Python, Ruby, JavaScript)



Programming languages by programming paradigms

- Imperative - computer is told step by step what to do (how to do the task?) (e.g. Pascal, Java, C)
- Procedural - subtasks broken into procedures (e.g. Java, Pascal)
- Modular - larger programs broken into procedures, but also modular sub tasks (e.g. Python)
- Object-oriented - data and processing procedures are combined into objects/classes (e.g. Java, C++, Python, R...)
- Declarative- the computer is told what to do, but not how (e.g. XSLT)

Errors and bugs

- programming errors are called bugs and the process of tracking them down and correcting them is called debugging.
- Errors – syntax, logic/semantic, runtime (undefined variable, dividing by zero, wrong operation on wrong datatype)

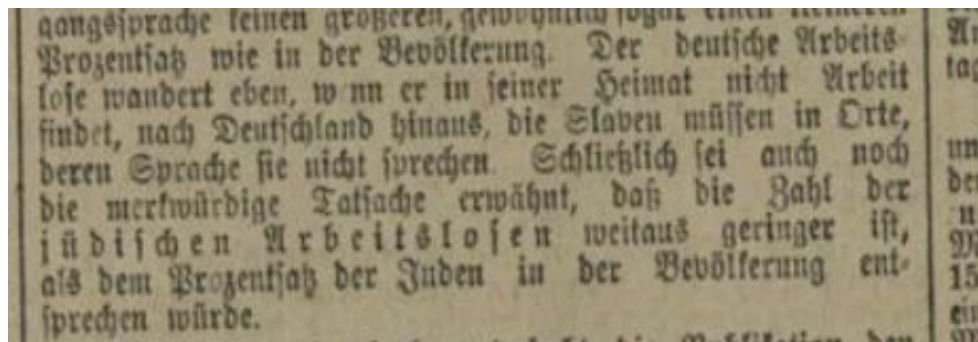


Programming in the DH

- Study large cultural collections: newspapers, books, parliamentary debates, correspondence, historical archives, etc
- Ask new research questions: computational methods make some questions feasible that would otherwise be practically impossible to investigate
- What can we study?
 - authorship, themes, genres, sentiment/emotion, entities, discourse, semantic change, historical language, multilingual corpora, low-resource languages, historical sources, character analysis, linguistic creativity, development of philosophical ideas, public discourse, stereotypes...

Challenges in the DH

- historical language: spelling, grammar, meaning and language change
- OCR / HTR errors
- non-standard language
- metaphors, satire, irony, sarcasm, poetic language
- small datasets, low-resource languages
- complex data
- domain mismatch
- unstructured data



Questions, comments?